[Institution / department logo]

TALENTLENS: AI-ASSISTED RESUME SCREENING

Final Project Review | Department of Computer Science / IT Department | [Add Institution Name]

TEAM MEMBERS

2420030651 – SANJANA NANNAPANENI

2420030018 – SAI VIVEK MUNNURU

2420030489 – NAMAN

Guide: [Add Guide Name & Designation] | Academic Year: [Add Academic Year]

Introduction & Problem Statement

Background

Recruiters often review many resumes for each vacancy. Manual screening can be time-consuming and inconsistent. TalentLens helps organise the first-pass review using job requirements and resume text.

Problem Statement

The project addresses the time and effort required to compare candidate resumes against a job description and required skills.

Motivation / Need

Recruiters and hiring teams benefit from faster, more consistent first-pass screening. Manual review can overlook relevant skills or spend time on poor matches.



Objectives & Scope

Objectives

1. Develop a local web interface for resume screening.
2. Extract text from supported PDF, DOCX and TXT resumes.
3. Compare resume content with a job title, description and selected skills.
4. Present screening results in a readable interface.

Literature Survey

S.No	Author(s) & Year	Method / Technique	Key Findings	Limitation / Gap
1	[Author, Year]	[Method used]	[Main result]	[Gap or limitation]
2	[Author, Year]	[Method used]	[Main result]	[Gap or limitation]
3	[Author, Year]	[Method used]	[Main result]	[Gap or limitation]
4	[Author, Year]	[Method used]	[Main result]	[Gap or limitation]



Research Gap

Project-specific literature references and a verified comparison with existing systems still need to be added. The intended gap is a simple, local prototype that combines resume text extraction, recruiter-selected skills and optional AI-assisted analysis.

Proposed System & Methodology

STEP 1

Define Problem

Identify the need, scope and success criteria.

STEP 2

Collect Data

User-supplied resumes in PDF, DOCX or TXT format; no fixed benchmark dataset is bundled.

STEP 3

Design

HTML/CSS/JavaScript frontend; FastAPI backend; document text extraction; keyword matching; optional OpenAI API integration.

STEP 4

Implement

Frontend form, resume upload, API endpoint, text extraction, skill matching and results display.

STEP 5

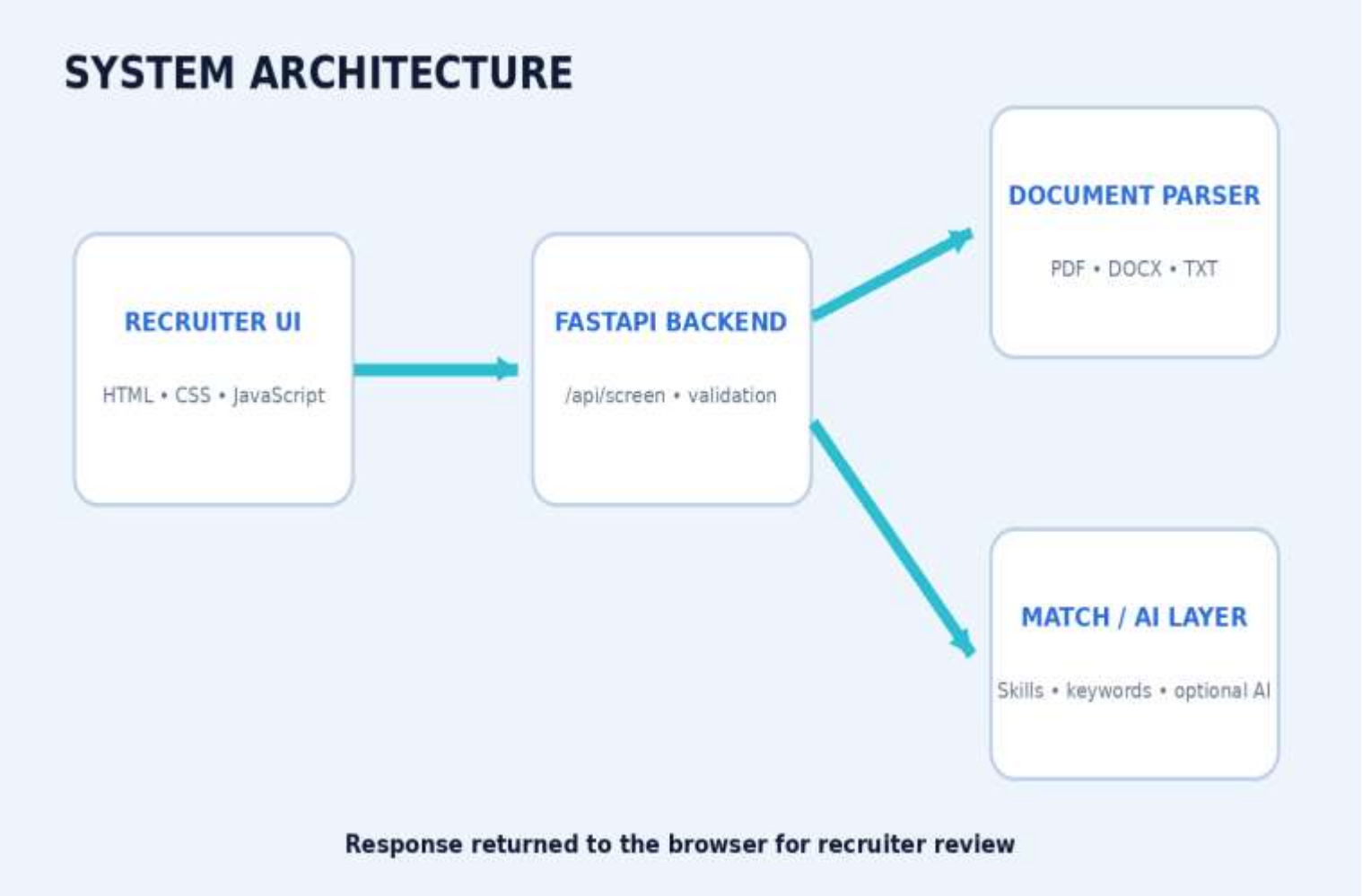
Test & Evaluate

Validate upload formats, required job fields, API health and screening response using sample resumes.

Key Novelty / Contribution

TalentLens combines a simple recruiter-controlled skill selector with local resume processing and optional AI assistance. Its current role is decision support for an initial review, not an automated hiring decision.

System Architecture



Components

1. **Recruiter Interface**
   Collects role details, skills and resume files.

2. **Upload & API**
   Collects role details, skills and resume files.

3. **Text Extraction**
   Collects role details, skills and resume files.

4. **Screening & Results**
   Collects role details, skills and resume files.

Hardware & Software Requirements

Hardware

Standard laptop/desktop; dedicated GPU not required
Local disk for source code and temporary uploads
Not applicable — software-only project
Localhost access; keyboard and display

Software & Tools

- Windows (development environment)
- Python (version depends on local installation)
- Visual Studio Code
- GitHub: add repository URL if available

Libraries / Frameworks

- FastAPI / Uvicorn
- Document parsing libraries for PDF and DOCX
- Optional OpenAI API (requires API key)
- No database required for the current local MVP

Deployment / Platform: Local machine — Uvicorn on 127.0.0.1:8002

Dataset / Data Understanding

N/A

Records (rows)

N/A

Features (columns)

[0]

Missing values

[2]

Target classes

- Source: User-supplied sample resumes
- Features: Inputs: job title, job description, selected skills and resume text. Output: screening/match information.
- Quality: No fixed labelled dataset; evaluation should use a documented set of test resumes.



Data Preprocessing & Exploratory Analysis

No labelled tabular dataset in current MVP

Not applicable: no trained feature matrix

Cleaning

Not applicable as a conventional tabular dataset; unsupported or unreadable files should be reported.

Transformation

Extract document text and normalise it for matching; optional AI mode analyses extracted text.

Feature Selection

Job description and recruiter-selected skills guide matching; no trained feature-selection model is used in the current MVP.

Module-wise Description

1 Recruiter Input

Purpose: Processes its assigned input and returns the next workflow output
Input → Output: Job fields → validated request
Developed by: Assign based on actual contribution

2 Resume Upload & Extraction

Purpose: Processes its assigned input and returns the next workflow output
Input → Output: Job fields → validated request
Developed by: Assign based on actual contribution

3 Skill Matching

Purpose: Processes its assigned input and returns the next workflow output
Input → Output: Job fields → validated request
Developed by: Assign based on actual contribution

4 Results & Optional AI

Purpose: Processes its assigned input and returns the next workflow output
Input → Output: Job fields → validated request
Developed by: Assign based on actual contribution

Algorithms & Technologies Used

Document text extraction

Why chosen
Converts supported resumes into text for downstream comparison.

Key parameters
Supported formats: PDF, DOCX and TXT

Keyword / skill matching

Why chosen
Converts supported resumes into text for downstream comparison.

Key parameters
Supported formats: PDF, DOCX and TXT

Optional AI-assisted analysis

Why chosen
Converts supported resumes into text for downstream comparison.

Key parameters
Supported formats: PDF, DOCX and TXT

Selection criteria: [How the final approach was chosen – metric, cost, speed, simplicity]

Performance values are pending. Run and document repeatable tests before reporting metrics.

Feature Selection

Job title, job description, recruiter-selected skills and extracted resume text.

Preprocessing Pipeline

Validate inputs → read uploaded files → extract text → compare against role requirements → format results.

Data Splitting

Not applicable: the current MVP does not train a supervised model.

Configuration

Local server port 8002; optional AI mode depends on OPENAI_API_KEY and configured model.

Code snippet / pseudocode

```
Screening flow (pseudocode): validate job fields;
extract resume text; compare skills; optionally
request AI analysis; return results.
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2,
                     random_state=42)

pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", Model())
])
pipeline.fit(X_train, y_train)
```

Working Prototype / Output Screens

[Insert screenshot of job details and skill selector]

Recruiter Input
Result summary after a successful API response.

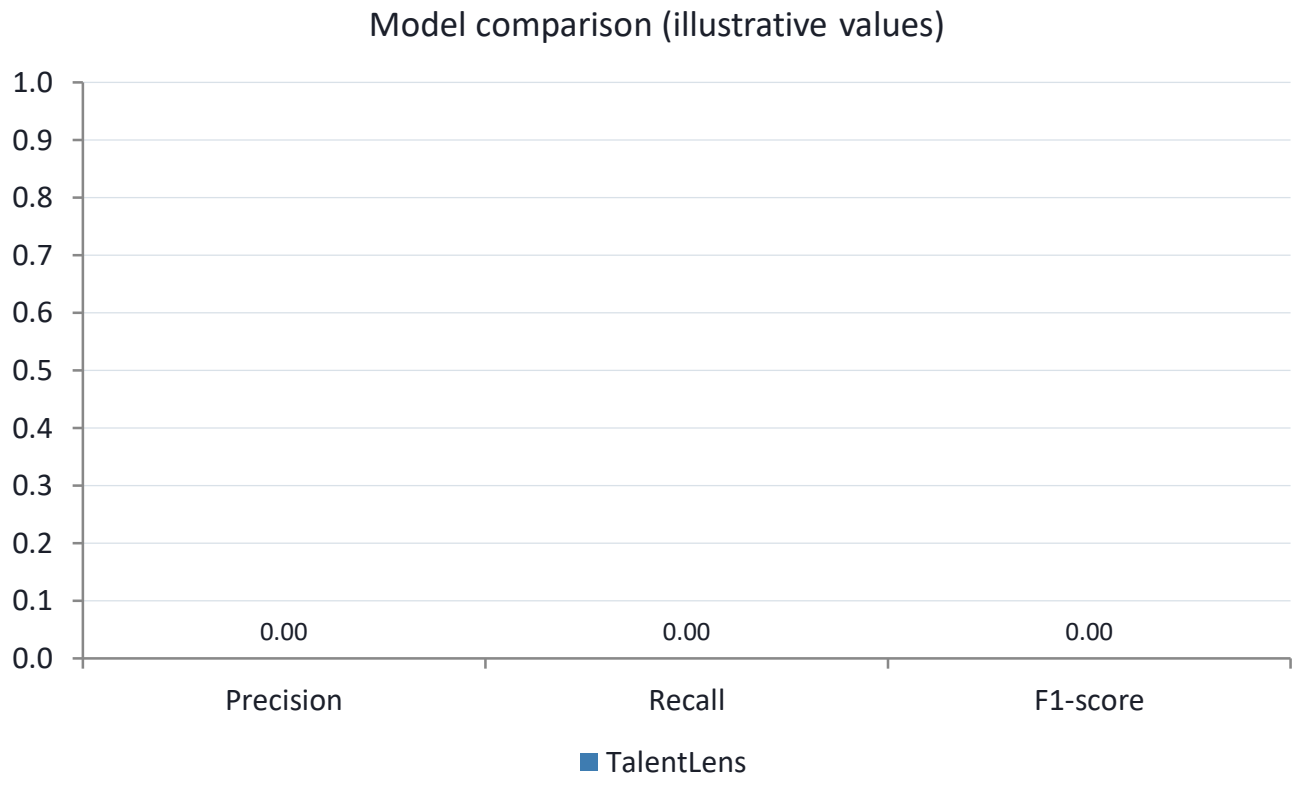
[Insert screenshot of resume upload]

Resume Upload
Result summary after a successful API response.

[Insert screenshot of screening results]

Screening Results
Result summary after a successful API response.

Results & Performance Comparison



Metric	Model A	Model B	Model C
Accuracy	0.00	0.00	0.00
Precision	0.00	0.00	0.00
Recall	0.00	0.00	0.00
F1-score	0.00	0.00	0.00
ROC-AUC	0.00	0.00	0.00

Best: Benchmark comparison pending; do not report illustrative template values as project results.

No validated benchmark results are available yet. Replace placeholders after testing.

Analysis & Key Findings

No trained classifier or labelled test set; use documented sample-based evaluation

Not measured

Best [metric] achieved by TalentLens prototype

Interpretation: Use only after running and documenting repeatable tests.
Error analysis: Potential issues include scanned PDFs without OCR, inconsistent resume formatting and keyword mismatch.
Key drivers: Job description, selected skills and quality of extracted text.

Testing & Validation

Test ID	Test Description	Expected Result	Actual Result	Status
TC-01	Upload a supported DOCX resume	Text is extracted and screening response is returned	Record after running the test	Pending
TC-02	Upload a supported DOCX resume	Text is extracted and screening response is returned	Record after running the test	Pending
TC-03	Upload a supported DOCX resume	Text is extracted and screening response is returned	Record after running the test	Pending

Functional Testing

Formal test execution and outcomes must be recorded by the team.

Validation Method

Not applicable to a trained model; use a documented sample-based evaluation.

Performance Testing

Measure response time and success/failure across representative file types.

Challenges & Limitations

Challenges Faced

- Local server port conflict → use port 8002 for the current project.
- Resume format differences → support PDF, DOCX and TXT extraction.
- Optional AI dependency → core screening can run without an API key.

Limitations

- Scanned PDFs are not OCR-processed in the current MVP.
- No trained or validated ranking model is included.
- Current MVP is local and not production-hardened.

Conclusion

1. A local web prototype for resume upload, text extraction and job-skill matching was developed using a FastAPI backend.

2. Quantitative performance results have not yet been established; report only results from completed tests.

3. The prototype provides a workflow for initial resume review; human recruiters must make final decisions.



Final Outcome

Pending

Working local screening workflow
Objectives achieved: Report after validation

Future Scope & Applications

Improve Performance
Evaluate match scoring and optional AI output against human-reviewed examples.

Expand Data / Features
Test diverse resume layouts, roles and skill vocabularies.

Deploy & Build an Interface
Improve the web dashboard and add exportable reports.

Scale & Secure
Add authentication, secure storage, privacy controls and deployment hardening before wider use.

Applications

- Recruitment teams
- Campus placement cells
- Small organisations screening applicants
- Recruiters and placement coordinators

Team Contribution

Reg. No.	Name	Contribution / Modules Handled
2420030651	Sanjana Nannapaneni	Planning, interface review and documentation — confirm actual tasks.
2420030018	Sai Vivek Munnuru	Backend/API workflow review and integration — confirm actual tasks.
2420030489	Naman	Testing, presentation and documentation — confirm actual tasks.
—	—	—

References

[1] A. Author, B. Author, “Title of the paper,” Journal Name, vol. X, no. Y, pp. 1–10, Year.

[2] A. Author, Title of the Book. City, Country: Publisher, Year.

[3] Organisation, “Dataset or website title,” Year. [Online]. Available: URL

[4] A. Author, “Title of the conference paper,” in Proc. Conference Name, Year, pp. 1–8.

[5] Add verified papers, framework documentation and API references before submission.

Project Links

GitHub: [Add verified URL]
Dataset: [Add verified URL]
Demo video: [Add verified URL]

THANK YOU

Questions & Discussion

[Add team contact if required]

[Guide Name / Department]